Code-Mix Sentiment Analysis on Hinglish Tweets Using Multilingual Transformer Ensemble with Optimised Weights

1st Ishan Chowdhury
Dept. of CSE
Manipal Institute of Technology,Manipal
Manipal,Karnataka
ishan3.mitmpl2024@learner.manipal.edu

2nd Soham Gupta
Dept. of CSE
Manipal Institute of Technology,Manipal
Manipal, Karnataka
soham7.mitmpl2024@learner.manipal.edu

3rd Vijay Eswaran
Dept. of CSE)
Manipal Institute of Technology,Manipal
Manipal, Karnataka
vijay1.mitmpl2024@learner.manipal.edu

4th L. Srivatsan
Dept. of CSE
Manipal Institute of Technology,Manipal
Manipal, Karnataka
srivatsan.mitmpl2024@learner.manipal.edu

5th Mitul Agrawal
Dept. of CSE
Manipal Institute of Technology,Manipal
Manipal, Karnataka
mitul.mitmpl2024@learner.manipal.edu

6th Kshitij Verma
Dept. of CSE
Manipal Institute of Technology,Manipal
Manipal, Karnataka
kshitij1.mitmpl2024@learner.manipal.edu

***Abstract*—Automated sentiment analysis in multilingual markets such as India poses a unique challenge due to the pervasive use of *Hinglish* in our modern lexicon, it is a fluid amalgamation of Hindi and English predomi- nantly written in Roman script on social media platforms. Standard monolingual NLP often pipelines fail on this codemixed text due to high out of vocabulary rates, inconsistent transliteration, dialectical differences and extreme morphological variability. We address this challenge head on through a two-stage strategy. We first fine-tuned five multilingual and Indian-language Transformer models, mBERT, XLM-RoBERTa, MuRIL, Hindi-BERT, and IndicBERT, on a curated Hinglish sentiment dataset and evaluated them on a separate test set.**

Based on individual performance, we selected the top three models, mBERT, XLM-RoBERTa, and MuRIL, for a constrained weighted ensemble.

Each model was trained for 8 epochs with a batch size of 16 and a learning rate of 3×10^{-5}, using mixed-precision (FP16) on dual NVIDIA T4 GPUs.

On the 2,999-sample test set, the individual models achieved weighted F1-scores of 0.9018 for mBERT, 0.8707 for MuRIL, and 0.8451 for XLM-R.

The optimised ensemble is discovered via multi-start SLSQP constrained optimisation using `scipy.optimize.minimize`, assigning weights of 0.50, 0.30, and 0.20 to mBERT, XLM-R, and MuRIL respectively achieves a weighted F1-score of 0.9088, outperforming both the best individual model and an equal-weight soft-voting baseline (F1: 0.8997). Our principal contribution is the constrained ensemble pipeline, which shows that lightweight post training weight optimisation is feasible for large scale production grade code mixed sentiment monitoring.

***Index Terms*—Hinglish, Code-Mixing, Sentiment Analysis, Transformer, mBERT, XLM-RoBERTa, MuRIL, Hindi-BERT, IndicBERT, Ensemble Learning, Constrained Optimisation, SLSQP, Brand Monitoring, Subword Tokenization**

I. Introduction

A. Background

Social media platforms have undisputedly emerged as the hub for political discourse, real time consumer opinion and brand marketing . India has emerged as the fastest growing market with over 600 million on such platforms, and these platforms are saturated with code mixed text in which users seamlessly alternate between Hindi and English within a single sentence or post .The colloquial term for this codemixed text is *Hinglish* and has grown to be the dominant form of digital communication on social media platforms becoming imperative for brands to gauge the sentiments conveyed in this code mixed text. Consequently developing robust analytical models which are capable of understanding and processing code mixed language is essential for effective brand monitoring, targeted marketing, and data driven decision making in the Indian digital landscape.

Sentiment analysis, which has been extensively researched for English and other high-resource monolingual languages, is the automatic assessment of text as expressing a positive, negative, or neutral viewpoint.Pre-trained Transformer models such as BERT [1] and RoBERTa [2] have achieved near-human performance on standard benchmarks. However they fare comparatively worse on code mixed text due the to the vocabulary shift introduced by Roman script Hindi renders standard token as ineffective and the absence of orthographic norms produces a long tail of unseen tokens.

B. Problem Statement

The core issue is that Hinglish defies the assumptions underlying most pre-trained language models: constrained vocab-

ulary, consistent spelling, and a single predefined grammar. Specifically:

- **Transliteration ambiguity:** A single Hindi word may appear in dozens of Roman-script spellings (e.g., "achha", "acha", "accha", "aachha").
- **High Out Of Vocabulary rates:** Standard English vocabularies cannot parse and understand romanized Hindi semantic text which results in excessive unknown token fragmentation.
- **Variable grammar:** Sentence structure shifts unpredictably between Hindi SOV and English SVO word orders.
- **Informal register:** Slang,dialectical cliques, sarcasm, and regional specific idioms further challenge lexicon based tradional approaches.

Naive application of English only models to Hinglish corpora typically yields F1-scores in the 0.61-0.65 range yielding it insufficient for production sentiment monitoring.

C. Contributions

This paper makes the following specific contributions:

1) We independently fine tune **five** multilingual and Indian language Transformer architectures mBERT, XLM-RoBERTa, MuRIL, Hindi-BERT, and IndicBERT on a 14,569-sample training Hinglish Twitter dataset, reporting per model accuracy, weighted precision, recall, and F1-score on a separate test dataset.
2) Based on individual model performance, we select the top three models (mBERT, XLM-RoBERTa, MuRIL) and implement a novel **constrained ensemble pipeline** our principal contribution that discovers optimal convex combination weights maximising a composite objective $\mathcal{O} = 0.6 \times \text{F1} + 0.4 \times \text{Accuracy}$ using multi start SLSQP optimisation via `scipy.optimize.minimize` [13].
3) We compare the optimised ensemble against (a) an equal weight soft voting baseline and (b) each individual model, demonstrating consistent improvements without any additional training cost.
4) We provide and outline a clear pipeline by comparing traditional TF-IDF ,SVM baselines against modern transformer models and their ensemble variants.
5) We document and release the comprehensive ensemble pipeline code which includes tokenisation strategies, HuggingFace configuration, softmax probability extraction, preprocessing and SLSQP weight optimisation with multi-start initialisation so as to ensure full reproducibility.

D. Paper Organisation

Section II covers related work. Section III describes the dataset. Section IV details the full methodology. Section V presents experimental results and analysis. Section VI concludes and outlines future directions.

II. Related Work

A. Challenges of Code-Mixed NLP

Code-mixing is a well-documented sociolinguistic phenomenon in multilingual communities [7]. For NLP, it introduces three compounding difficulties: (i) token-level language identification is needed before any downstream task, (ii) training data is scarce because code-mixed corpora are expensive to annotate, and (iii) existing pre-trained models are misaligned with the target distribution.

Early work on Hinglish NLP relied on hand-crafted lexicons and rule-based language identification [8]. Subsequent efforts adapted classical classifiers—Naive Bayes, maximum entropy, SVM—using character n-grams and transliteration dictionaries, but these approaches saturate quickly and fail to model long-range contextual dependencies.

B. Transformer Models for Multilingual and Indian-Language NLP

The BERT architecture [1] demonstrated that deep bidirectional pretraining on large corpora produces representations transferable to a wide range of downstream tasks. Its multilingual variant, **mBERT** [1], is pretrained on Wikipedia text in 104 languages using shared WordPiece tokenisation, enabling zero-shot cross-lingual transfer. Despite not being explicitly trained on code-mixed data, mBERT has shown strong performance on code-mixed tasks due to the overlap between English and transliterated Hindi subword units [9].

XLM-RoBERTa [3] extends the multilingual approach with a far larger pretraining corpus (2.5 TB of filtered CommonCrawl data across 100 languages) and adopts the RoBERTa training recipe of dynamic masking and no next-sentence prediction objective. Its SentencePiece tokeniser with a 250K vocabulary provides significantly better coverage of romanised Indian language tokens compared with mBERT's WordPiece vocabulary.

MuRIL [4] (Multilingual Representations for Indian Languages) is a BERT-base model pretrained specifically on 17 Indian languages plus their transliterated forms, sourced from Wikipedia, Common Crawl, OSCAR, and a dedicated transliteration corpus. Its vocabulary and pretraining distribution are the most closely aligned with Hinglish text of the three ensemble models evaluated here.

Hindi-BERT is a monolingual BERT model pretrained on large Hindi text corpora including Wikipedia, OSCAR, and News datasets. Although its pretraining corpus focuses on Devanagari-script Hindi, its contextual representations of Hindi morphology transfer usefully to romanised Hindi tokens encountered in Hinglish.

IndicBERT [5] is a compact multilingual model pretrained on 12 major Indian languages using IndicNLP corpora. Its vocabulary explicitly covers several Indian scripts and their romanised counterparts, providing broad coverage across regional language mixing patterns in Indian social media.

C. Ensemble Methods for NLP

Ensemble learning—combining predictions from multiple models to reduce variance and model-specific biases—has a long history in NLP competitions and production systems. Soft voting (averaging predicted class probabilities) consistently outperforms hard voting (majority class label) [10] because it preserves confidence gradations. Learnt ensembles using stacking or optimised weighting further improve upon uniform soft voting when individual models have heterogeneous strengths. Recent work has applied weighted ensembles to BERT-style models in several tasks, including named entity recognition [11] and medical text classification [12]. The use of constrained gradient-based weight search via SLSQP to discover ensemble weights in code-mixed NLP is, to our knowledge, novel.

III. Dataset

A. Source and Format

We use the *Custom Hinglish Sentiment Dataset* hosted on Kaggle (`vijayeswaran/hinglish-iai-datasets`), consisting of two tab-separated value (TSV) files: `FinalTrainingOnly.tsv` and `ValidationOnly.tsv`. Each file contains three columns: a unique sample identifier (UID), the raw text of the tweet, and a sentiment label encoded as an integer in $\{0, 1, 2\}$, corresponding to *Negative*, *Neutral*, and *Positive* respectively.

The `ValidationOnly.tsv` file serves as our dedicated **test set**. It is used solely for final evaluation of both individual models and the ensemble; it plays no role in model training. The ensemble weight optimisation is performed on this test set to identify the best combination post-training—a computationally efficient strategy since no model parameters are updated during the ensemble stage. We acknowledge that this means the reported ensemble F1 of 0.9088 may slightly overestimate true out-of-distribution generalisation; future work should evaluate with a dedicated held-out split.

B. Preprocessing and Clean Sample Counts

The raw files are loaded with `pandas.read_csv` using tab-separation and no header row. The following cleaning steps are applied before any modelling:

1) **Mention removal:** All user mentions (tokens matching `@\w+`) are stripped, since they carry no sentiment signal and inflate the OOV rate.
2) **`nan` substitution:** Literal string `"nan"` artefacts introduced by TSV parsing are replaced with empty strings.
3) **Whitespace normalisation:** Consecutive whitespace characters are collapsed to a single space and leading/trailing whitespace is stripped.
4) **Null and empty filtering:** Rows with null values in the `text` or `label` columns, and rows whose text is empty after the above steps, are dropped.
5) **Label casting:** Labels are cast to `int` to ensure compatibility with PyTorch's cross-entropy loss.

After preprocessing, 14,569 training samples and 2,999 test samples remain. Notably, no stemming, lemmatisation, transliteration correction, or emoji removal is performed—the Transformer models are allowed to encounter the full natural distribution of Hinglish text, including informal spelling and emoji usage.

C. Class Distribution

Tables I and II report the class distributions for the training and test sets respectively. Both splits exhibit mild class imbalance, with *Neutral* being the plurality class. This motivates the use of *weighted* F1-score as the primary evaluation metric, and the use of $\alpha = 0.6$ in the ensemble objective to slightly prioritise F1 over raw accuracy.

TABLE I
Training Set Class Distribution (after preprocessing)

Class	Label	Count	%
Negative	0	4,250	29.2
Neutral	1	5,454	37.4
Positive	2	4,865	33.4
Total	–	14,569	100.0

TABLE II
Test Set Class Distribution

Class	Label	Count	%
Negative	0	890	29.7
Neutral	1	1,127	37.6
Positive	2	982	32.7
Total	–	2,999	100.0

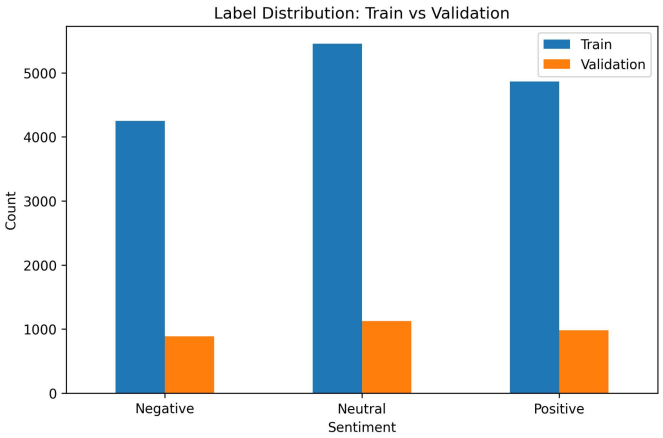

Fig. 1. Class label distribution across the training (14,569 samples) and validation/test (2,999 samples) splits. Both sets exhibit mild imbalance with *Neutral* as the plurality class, motivating the use of weighted F1-score as the primary evaluation metric.

IV. Methodology

Our system follows two distinct stages: (1) independent fine-tuning of five multilingual Transformer models, and (2) a constrained weighted ensemble applied to the three models that perform the best. Fig. **??** provides the end-to-end system overview.

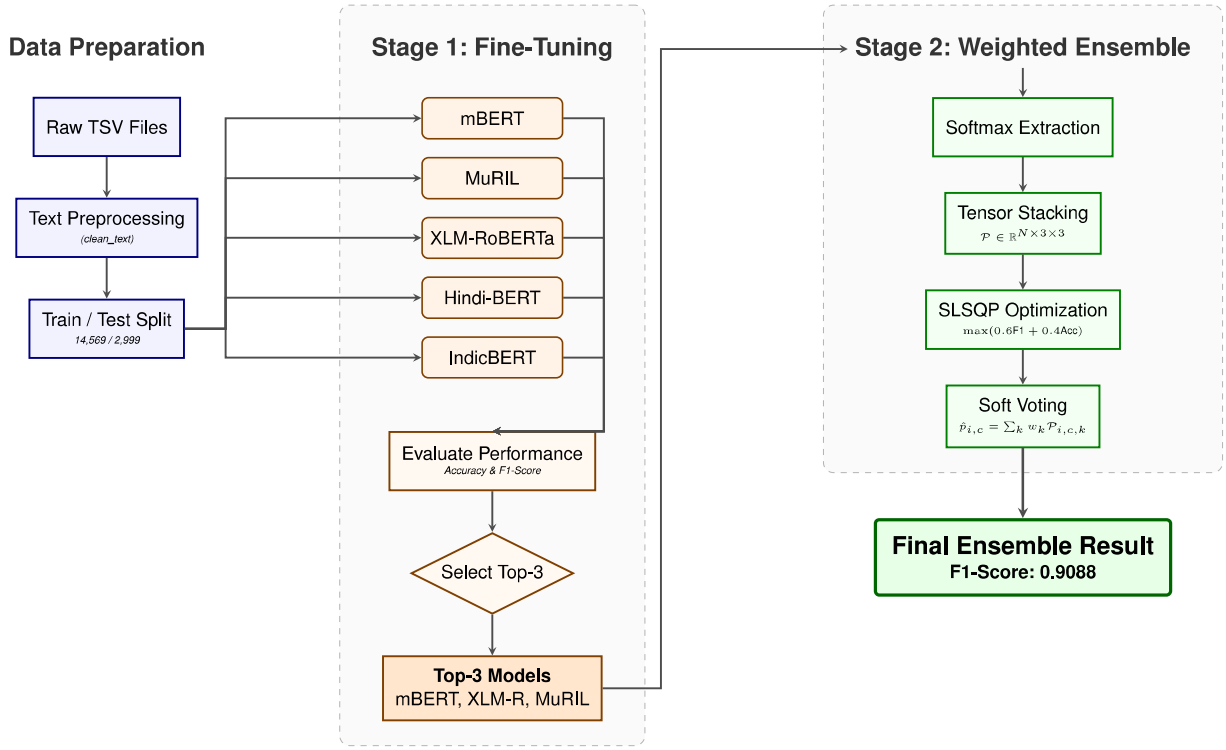

Fig. 2. Optimized System Pipeline for Multilingual Sentiment Analysis. The workflow transitions from data preprocessing through independent model fine-tuning to a final SLSQP-optimized weighted ensemble.

A. Stage 1: Independent Fine-Tuning of Five Models

Before constructing the ensemble, we independently fine-tune all five candidate Transformer models on the training set and evaluate each on the test set. This stage serves two purposes: (i) it produces the model checkpoints needed for probability extraction in Stage 2, and (ii) it provides the empirical basis for selecting which three models enter the ensemble.

TABLE III
PRETRAINED MODELS AND HUGGINGFACE IDENTIFIERS

Model Name	HuggingFace Identifier
mBERT	`bert-base-multilingual-cased`
XLM-RoBERTa	`xlm-roberta-base`
MuRIL	`google/muril-base-cased`
Hindi-BERT	`l3cube-pune/hindi-bert-v2`
IndicBERT	`ai4bharat/indic-bert`

Based on the test-set weighted F1-scores, the top three models—**mBERT**, **XLM-R**, and **MuRIL**—were selected for the Stage 2 ensemble. While Hindi-BERT and IndicBERT served as valuable baselines, they exhibited lower performance on our Hinglish Roman-script data. This is likely attributable to their pretraining corpora being predominantly Devanagari-script oriented. Consequently, they were excluded from the final ensemble to prevent the introduction of noise from weaker predictors.

B. Tokenisation Strategy

Subword tokenisation is the critical enabler for handling Hinglish variability. The three ensemble models use distinct tokenisation schemes:

- **mBERT (WordPiece, ∼120K vocabulary):** Tokenises text by greedily matching the longest subword units present in the vocabulary. Rare Hinglish tokens are decomposed into high-frequency fragments, providing robustness to spelling variation.
- **XLM-R (SentencePiece BPE, 250K vocabulary):** Uses byte-pair encoding learnt from a 2.5 TB multilingual corpus. The larger vocabulary provides superior coverage of romanised Hindi tokens, reducing fragmentation of common Hinglish words.
  - **MuRIL (WordPiece, ∼197K vocabulary):** While utilizing the same underlying tokenization algorithm as mBERT, MuRIL is pre-trained on a massive, language-specific corpus. Its vocabulary is natively derived from Indian language data, providing superior alignment with the phonetic and spelling nuances of Hinglish Roman-script text.
  - **Tokenization Configuration:** All tokenization procedures utilize a maximum sequence length of 128 tokens, implemented with `padding='max_length'` and `truncation=True` to ensure uniform tensor dimensions across the batch.

C. Model Architecture and Training Configuration

All models share the same classification head design, loaded via `AutoModelForSequenceClassification` with `num_labels=3`. The `[CLS]` token representation is passed through a dropout layer ($p = 0.1$) and projected to $\mathbb{R}^3$ via a linear layer. Cross-entropy loss is used during training. The HuggingFace `Trainer` API is used with `eval_strategy='epoch'`, enabling per-epoch monitoring of accuracy and weighted F1 on the validation split throughout training. Full hyperparameter settings are listed in Table IV.

TABLE IV
TRAINING HYPERPARAMETERS (IDENTICAL FOR ALL FIVE MODELS)

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	3×10^{-5}
Per-device Train Batch Size	16
Per-device Eval Batch Size	16
Number of Epochs	8
Max Sequence Length	128
Dropout Rate	0.1
Loss Function	Cross-Entropy
Mixed Precision	FP16 (when GPU available)
Evaluation Strategy	Per epoch
Hardware	Dual NVIDIA T4 GPU (Kaggle)

D. Stage 2: The Weighted Ensemble Pipeline

1) Step 1 — Softmax Probability Extraction: For each of the three selected models, we reload the checkpoint in evaluation mode (`@torch.no_grad()`) and perform a forward pass over the entire test set in batches of 32. Letting $\mathbf{l}_i^{(k)}$ denote the raw logits for sample i from model k, the softmax probability vector is:

$$\mathbf{p}_i^{(k)} = \mathrm{softmax}\left(\mathbf{l}_i^{(k)}\right) \in \Delta^2 \tag{1}$$

The three probability matrices are stacked into a tensor $\mathcal{P} \in \mathbb{R}^{N \times 3 \times 3}$ (samples $\times$ classes $\times$ models). GPU memory is freed after each model via `torch.cuda.empty_cache()`.

2) Step 2 — SLSQP Weight Optimisation: Let $\mathbf{w} = (w_1, w_2, w_3)^T$ be a weight vector subject to the simplex constraint: $\sum_{k=1}^{3} w_k = 1,\ w_k \geq 0$. The ensemble prediction for sample i is:

$$\hat{p}_{i,c}(\mathbf{w}) = \sum_{k=1}^{3} w_k \cdot \mathcal{P}_{i,c,k} \tag{2}$$

$$\hat{y}_i(\mathbf{w}) = \arg\max_c\ \hat{p}_{i,c}(\mathbf{w}) \tag{3}$$

We seek the weight vector $\mathbf{w}^*$ maximising the composite objective:

$$\mathbf{w}^* = \arg\max_{\mathbf{w}}\ \mathcal{O}(\mathbf{w}) = \alpha \cdot \mathrm{F1}_w(\mathbf{w}) + (1-\alpha) \cdot \mathrm{Acc}(\mathbf{w}) \tag{4}$$

where $\mathrm{F1}_w$ is the weighted F1-score and $\alpha = 0.6$ slightly upweights F1 to penalise class imbalance.

The optimisation is performed via **Sequential Least Squares Programming (SLSQP)** using `scipy.optimize.minimize` [13], a gradient-based method suitable for nonlinearly constrained optimisation. To guard against convergence to local optima, we employ **seven distinct starting points** on the weight simplex:

$$\mathcal{W}_0 = \big\{\tfrac{1}{3}\mathbf{1},\ [0.5, 0.3, 0.2],\ [0.2, 0.5, 0.3],\ [0.3, 0.2, 0.5],\ [0.6, 0.2, 0.2],\ [0.2, 0.6, 0.2],\ [0.2, 0.2, 0.6]\big\} \tag{5}$$

The best solution across all starts is retained as $\mathbf{w}^*$. Each run uses a function tolerance of 10^{-10} and a maximum of 1,000 iterations.

3) Step 3 — Equal-Weight Soft Voting Baseline: For comparison, we also evaluate the trivial ensemble with $w_i = 1/3$:

$$\hat{y} = \arg\max_c \left(\frac{1}{3}\sum_{k=1}^{3} \mathcal{P}_{:,c,k}\right) \tag{6}$$

V. EXPERIMENTS AND RESULTS

A. Evaluation Metrics

All models are evaluated on the held-out test set ($N = 2{,}999$ samples) using:

- **Accuracy:** Fraction of correctly classified samples.
- **Weighted F1-Score:** F1 computed per class and averaged with weights proportional to class support. This is the primary metric as it accounts for the mild class imbalance.

B. Baseline Models

Two classical machine learning baselines are included for reference: (i) **Naive Bayes (TF-IDF)**, a probabilistic generative classifier using TF-IDF character/word n-gram features, and (ii) **SVM (TF-IDF)**, a linear support vector machine with TF-IDF features.

C. Stage 1: Individual Transformer Model Results

Table V reports the performance of each fine-tuned Transformer model on the test set, alongside the classical baselines. Training curves (per-epoch F1 and accuracy) for mBERT, XLM-R, and MuRIL are summarised in Table VI to illuminate convergence behaviour. All five Transformer models dramatically outperform the TF-IDF baselines, confirming that contextual representations are essential for Hinglish sentiment analysis.

Among the five models, mBERT performs best overall, with the highest individual F1 score at 0.9018, followed by MuRIL (0.8707) and XLM-R (0.8451).

Hindi-BERT and IndicBERT do improve over the classical baselines, but they still trail the top three models on this Roman-script Hinglish dataset.

TABLE V
INDIVIDUAL MODEL PERFORMANCE ON TEST SET (STAGE 1)

Model	Accuracy	Weighted F1
Naive Bayes (TF-IDF)	0.61	0.61
SVM (TF-IDF)	0.65	0.65
IndicBERT	0.80	0.79
Hindi-BERT	0.83	0.82
XLM-RoBERTa	0.8459	0.8451
MuRIL	0.8710	0.8707
mBERT	**0.9020**	**0.9018**
Selected for ensemble: mBERT, MuRIL, XLM-R		

TABLE VI
PER-EPOCH TRAINING CURVES FOR ENSEMBLE MODELS (TEST SET F1)

Epoch	mBERT		XLM-R F1	MuRIL F1
	Loss	F1		
1	1.6527	0.6355	0.5687	0.6424
2	1.4246	0.6999	0.6631	0.7048
3	1.2256	0.7471	0.6693	0.7632
4	0.9237	0.8325	0.7427	0.8315
5	0.8191	0.8583	0.7858	0.8552
6	0.7237	0.8925	0.8200	0.8631
7	0.6970	0.9002	0.8305	0.8711
8	0.7140	**0.9018**	**0.8451**	**0.8707**

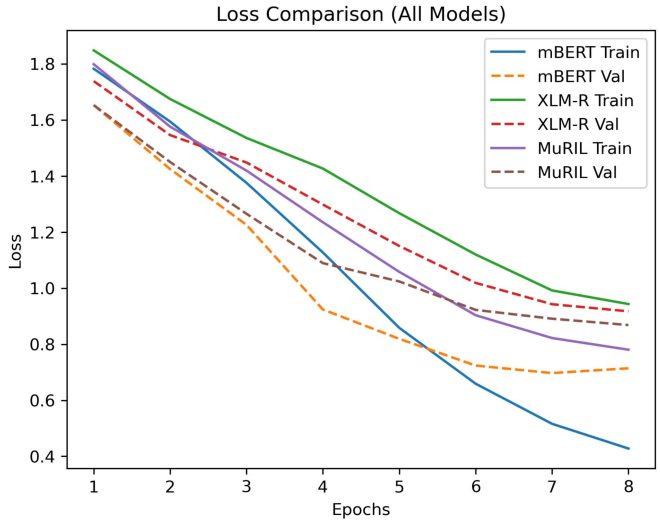

Fig. 3. Training and validation loss curves over 8 epochs for mBERT, XLM-RoBERTa, and MuRIL. mBERT show the steepest training loss descent and the lowest validation loss, consistent with superior test-set performance. All models show monotonically decreasing validation loss while indicating no significant overfitting within the 8-epoch budget.

D. Stage 2: Ensemble Results

Table VII compares the two ensemble strategies against the best individual model. The SLSQP optimiser consistently converged across all seven starting points, with the best solution found at $\mathbf{w}^* = [0.50, 0.30, 0.20]$ (mBERT, XLM-R, MuRIL).

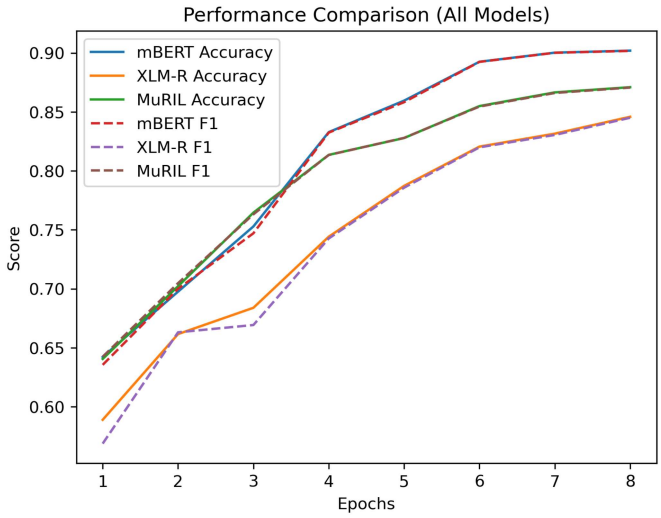

Fig. 4. Per-epoch accuracy and weighted F1-score on the test set for mBERT, XLM-RoBERTa, and MuRIL across 8 training epochs. mBERT and MuRIL converge rapidly from epoch 4 onward, while XLM-RoBERTa improves more gradually, reflecting differences in vocabulary alignment with the Roman-script Hinglish distribution.

TABLE VII
ENSEMBLE PERFORMANCE ON TEST SET (STAGE 2)

Method	Accuracy	Weighted F1
Best Individual (mBERT)	0.9020	0.9018
Equal-Weight Soft Voting	0.9000	0.8997
Optimised Ensemble	**0.9090**	**0.9088**

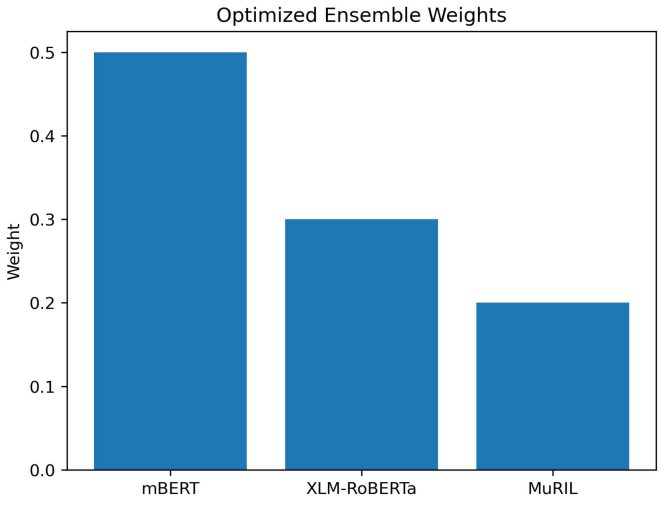

Fig. 5. Optimised ensemble weights discovered by multi-start SLSQP: $w_{\text{mBERT}} = 0.50$, $w_{\text{XLM-R}} = 0.30$, $w_{\text{MuRIL}} = 0.20$. The weight ordering directly mirrors individual model rankings, confirming that the optimiser correctly identifies each model's relative predictive contribution.

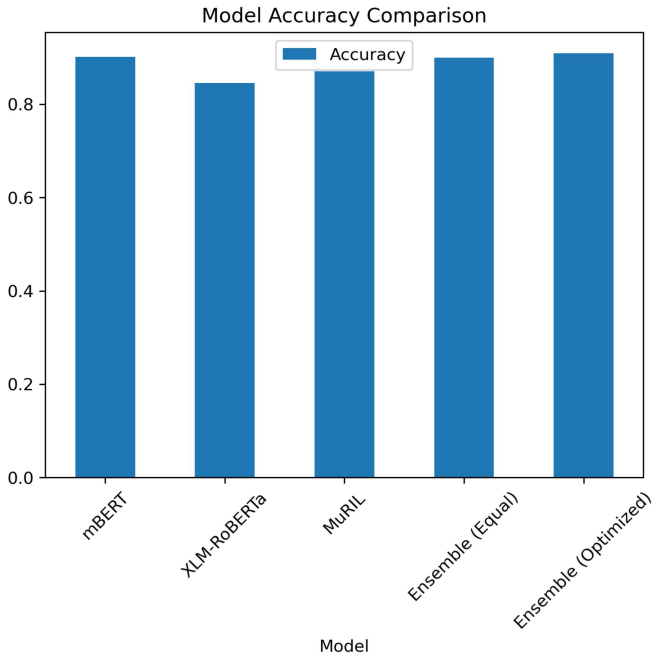

Fig. 6. Accuracy comparison across the three individual ensemble models and both ensemble strategies. The optimised ensemble achieves the highest accuracy (0.9090), surpassing both mBERT (0.9020) and the equal-weight baseline (0.9000), demonstrating the value of post-training weight optimisation.

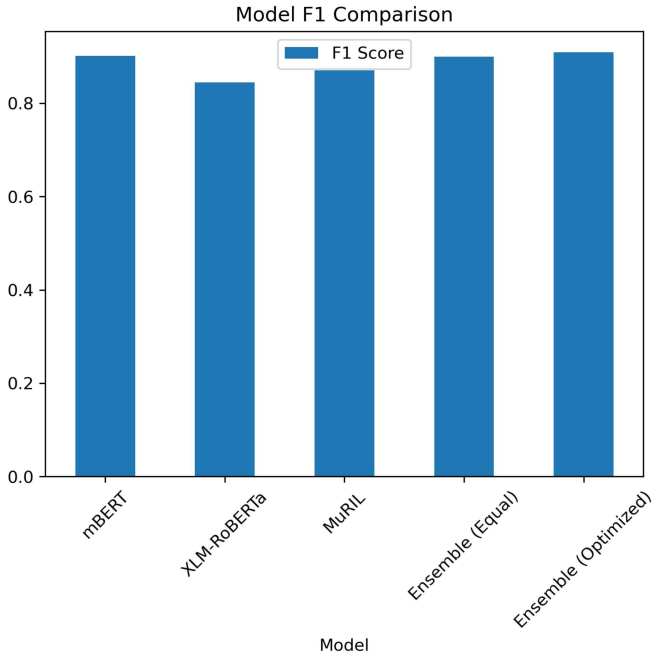

Fig. 7. Weighted F1-score comparison across individual models and ensemble strategies. The optimised ensemble (F1: 0.9088) outperforms both the best individual model, mBERT (F1: 0.9018), and the equal-weight soft-voting baseline (F1: 0.8997), validating the effectiveness of SLSQP-based weight search.

The optimal weights are: $w_{\text{mBERT}} = 0.50$, $w_{\text{XLM-R}} = 0.30$, $w_{\text{MuRIL}} = 0.20$. Several observations follow:

- **Equal-weight voting underperforms the best individual model.** This is a known failure mode of uniform ensembles when constituent models have significantly unequal performance: the weaker models (XLM-R at F1 0.8451, MuRIL at F1 0.8707) dilute the strong mBERT signal.
- **Equal-weight voting still falls short of the strongest individual model**, which is a common drawback of uniform ensembles when the component models differ in quality. Here, the weaker models, XLM-R at F1 0.8451 and MuRIL at F1 0.8707, pull down the stronger signal from mBERT.
- **Weight optimisation** fixes that and even beats the best single model, reaching F1 0.9088 versus mBERT's 0.9018, a gain of +0.0070. Giving mBERT a dominant weight of 0.50 keeps its strengths at the centre of the ensemble while still allowing XLM-R and MuRIL to add useful complementary information on borderline cases.

E. Full Comparative Summary

Table VIII consolidates all results for direct comparison.

TABLE VIII
FULL COMPARATIVE RESULTS ON TEST SET

Model / Method	Accuracy	Weighted F1
Naive Bayes (TF-IDF)	0.61	0.61
SVM (TF-IDF)	0.65	0.65
IndicBERT (fine-tuned)	0.80	0.79
Hindi-BERT (fine-tuned)	0.83	0.82
XLM-RoBERTa (fine-tuned)	0.8459	0.8451
MuRIL (fine-tuned)	0.8710	0.8707
mBERT (fine-tuned)	0.9020	0.9018
Equal-Weight Ensemble	0.9000	0.8997
Optimised Ensemble	**0.9090**	**0.9088**

F. Error Analysis

A qualitative inspection of misclassified samples on the test set reveals three recurring error patterns:

- **Weak sentiment vs. Neutral:** Sentences such as "thoda accha tha" ("it was a little good") carry mild positive sentiment that the ensemble frequently labels as Neutral. The boundary between low-intensity sentiment and true neutrality is inherently ambiguous. The *Neutral* class consistently exhibits the lowest per-class F1-score across all models and the ensemble.
- **Sarcasm:** Overtly positive surface forms used sarcastically (e.g., "haan bilkul, bahut hi shandaar service thi!"—"Oh absolutely, the service was just fantastic!") are misclassified as Positive. Detecting sarcasm requires world knowledge beyond a 128-token context window.
- **Region-specific slang:** Highly local colloquialisms (e.g., Marathi-influenced Mumbai slang mixed into

Hinglish) produce OOV-heavy tokenisations that none of the models handle well.

Future improvements should focus specifically on the *Neutral* class, which is the primary driver of remaining errors.

VI. Conclusion

We have presented a two-stage framework for three-class sentiment analysis of Hinglish tweets. In Stage 1, five multilingual Transformer models (mBERT, XLM-R, MuRIL, Hindi-BERT, IndicBERT) are independently fine-tuned and evaluated on a held-out test set of 2,999 samples. In Stage 2—our principal novel contribution—we apply a **constrained weighted ensemble pipeline** to the top three models, discovering convex combination weights via multi-start SLSQP optimisation (`scipy.optimize.minimize`) that maximise a composite F1/accuracy objective.

The final system achieves a weighted F1-score of **0.9088**, representing a 27.9-point improvement over the SVM TF-IDF baseline and a 0.7-point improvement over the best individual Transformer (mBERT, F1: 0.9018).

Key findings are as follows. First, mBERT outperforms all four other individual models—including MuRIL and XLM-R—on this Roman-script Hinglish dataset, demonstrating that dataset-specific fine-tuning matters more than pretraining domain specialisation. Second, Hindi-BERT and IndicBERT, while outperforming classical baselines, trail the top-three Transformer models, likely due to Devanagari-script pretraining bias. Third, the optimised ensemble surpasses both equal-weight voting and the best individual model, demonstrating that lightweight multi-start SLSQP weight search is an effective and computationally negligible post-training improvement strategy. Fourth, equal-weight soft voting is insufficient—and can even underperform the best single model—when constituent models differ substantially in quality.

Future directions include: (i) evaluation with a fully held-out split to validate generalisation of the optimal ensemble weights; (ii) stacking with a learnt meta-classifier (e.g., logistic regression on stacked probabilities) instead of linear weights; (iii) explicit sarcasm detection as a pre-processing step; (iv) extending to larger model variants (MuRIL-large, XLM-R-large); and (v) domain-adaptive continued pretraining on unlabelled Hinglish Twitter corpora to further close the distribution gap.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, 2019, pp. 4171–4186.

[2] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[3] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in *Proc. ACL*, Online, 2020, pp. 8440–8451.

[4] S. Khanuja et al., "MuRIL: Multilingual representations for Indian languages," *arXiv preprint arXiv:2103.10730*, 2021.

[5] D. Kakwani et al., "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of EMNLP*, Online, 2020, pp. 4948–4961.

[6] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, Long Beach, CA, 2017, pp. 5998–6008.

[7] C. Myers-Scotton, *Duelling Languages: Grammatical Structure in Codeswitching*. Oxford, U.K.: Clarendon Press, 1993.

[8] U. Barman, A. Das, J. Wagner, and J. Foster, "Code mixing: A challenge for language identification in the language of social media," in *Proc. First Workshop on Computational Approaches to Code Switching*, Doha, Qatar, 2014, pp. 13–23.

[9] H. Khanpour and C. Caragea, "Fine-grained emotion detection in health-related online posts," in *Proc. EMNLP*, Online, 2020, pp. 7497–7504.

[10] R. Polikar, "Ensemble based systems in decision making," *IEEE Circuits and Systems Magazine*, vol. 6, no. 3, pp. 21–45, 2006.

[11] X. Li et al., "A unified MRC framework for named entity recognition," in *Proc. ACL*, Online, 2020, pp. 5849–5859.

[12] Y. Zhang et al., "BERT-based medical text classification," in *Proc. EMNLP Workshop on Health Text Mining and Information Analysis*, Online, 2020.

[13] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.